# Project Development

## Project Title

**Implement Client Script & UI Policy (Incident)**

## 1. Introduction

The Project Development phase involves implementing the planned UI Policy and Client Scripts in the ServiceNow Incident table. The development focuses on controlling field behavior, automatically updating values, validating Incident records, and restricting unwanted list-based changes.

## 2. Development Environment

The project is developed using:

- **Platform:** ServiceNow
- **Module:** Incident Management
- **Table:** Incident
- **Technology:** JavaScript
- **Features:** UI Policy, UI Policy Actions, Client Scripts, Form Validation

## 3. UI Policy Development

A UI Policy named **High Impact Control** is created for the Incident table.

### Configuration

- **Name:** High Impact Control
- **Table:** Incident
- **Active:** True
- **Condition:** Impact is 1 – High
- **Reverse if false:** True

When the Impact field is set to High, the UI Policy is triggered.

## 4. UI Policy Action Development

A UI Policy Action is created under the **High Impact Control** UI Policy.

### Configuration

- **Field:** Urgency
- **Read-only:** True
- **Visible:** Unchanged

When the UI Policy condition is satisfied, the Urgency field becomes read-only.

# 5. onChange Client Script Development

An **onChange Client Script** is created to automatically set Urgency when Impact changes.

### Configuration

- **Name:** Auto set urgency for high impact
- **Table:** Incident
- **Type:** onChange
- **Field:** Impact
- **Active:** True

### Script

```
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue == '') {
        return;
    }

    if (newValue == '1') {
        g_form.setValue('urgency', '1');
        g_form.addInfoMessage(
            'Urgency set to High for High impact incident.'
        );
    }
}
```

This script automatically sets Urgency to High when Impact is changed to High.

# 6. onSubmit Client Script Development

An **onSubmit Client Script** is developed to prevent an Incident from being saved when Assigned To is missing for a High Impact incident.

### Configuration

- **Name:** Prevent save if Assigned To missing
- **Table:** Incident
- **Type:** onSubmit
- **Active:** True

### Script

```
function onSubmit() {
    if (g_form.getValue('impact') == '1' &&
        g_form.getValue('assigned_to') == '') {

        g_form.showErrorBox(
            'assigned_to',
            'Assigned To is mandatory for High impact incidents.'
```

```
        );

        return false;
    }

    return true;
}
```

The script displays an error and prevents submission when the required condition is not satisfied.

# 7. onCellEdit Client Script Development

An **onCellEdit Client Script** is developed to prevent direct State changes through Incident list editing.

### Configuration

- **Name:** Prevent state change via list edit
- **Table:** Incident
- **Type:** onCellEdit
- **Field:** State
- **Active:** True

### Script

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {

    alert(
        'State cannot be updated using list editing. Please open the
Incident.'
    );

    callback(false);
}
```

This prevents the State value from being changed through direct list editing.

# 8. Development Workflow

The implemented workflow is:

**Incident Form**
↓
**Check Impact**
↓
**Impact = High**
↓
**UI Policy Applied**
↓
**Urgency Read-only**
↓

**onChange Script Sets Urgency = High**
↓
**User Attempts to Save**
↓
**onSubmit Validation**
↓
**Assigned To Available?**
↓
**Save Incident**

For list editing:

**Incident List → Edit State → onCellEdit Script → Alert → Change Blocked**

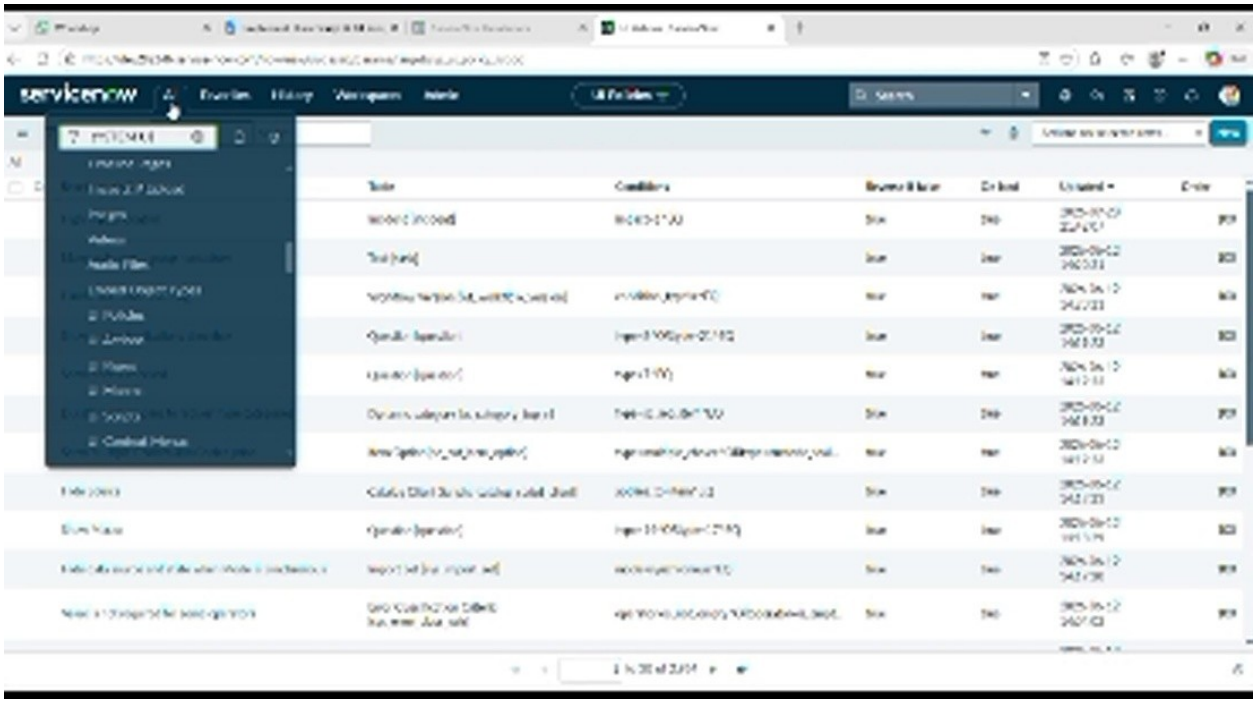
# 9. Development Outcome

The development phase implements the required controls:

- High Impact UI Policy
- Urgency UI Policy Action
- Automatic Urgency update
- Assigned To save validation
- State list-edit restriction

These components work together to enforce the defined Incident form rules.

**Screenshot**



# 10. Conclusion

The Project Development phase successfully defines the implementation of the **UI Policy, UI Policy Action, onChange Client Script, onSubmit Client Script, and onCellEdit Client Script** for the Incident table. The developed configuration provides dynamic field control, automatic value updates, save-time validation, and list-edit restrictions. The completed implementation is ready for the **Project Testing Phase**.